

ELEC6910J

Deep Reinforcement Learning

Lecture 7. Dynamic Programming (II) Policy Evaluation

Course Instructor: Ling PAN

Department of Electronic and Computer Engineering

Hong Kong University of Science and Technology

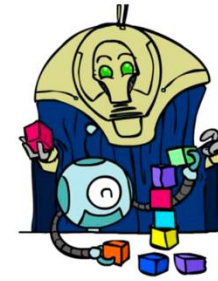
Most slides are adapted from UC Berkeley CS188/CS285, CMU 10-403

Announcements

- HW1 will be released next Monday

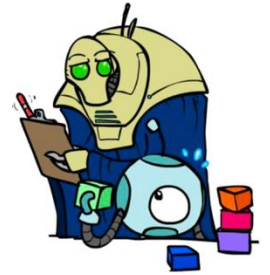
Project

- 1 student per project
 - Presentation: 30%; Report: 30%
- 15 min (Presentation: 13 min, Q&A: 2 min)
- Contents
 - Project topic
 - Research project on new algorithm / benchmark / application ...
 - Re-implementing / reproducing existing methods
 - Progress & findings so far
 - Plan until final report
- Schedule:
 - The presentation order will be randomly generated. If you have any scheduling conflicts, please discuss with our TAs in advance.
- Note: exam is scheduled on Oct. 23 (will have a review class before the exam)



Instruction

Grow knowledge,
collaborate,
work until success

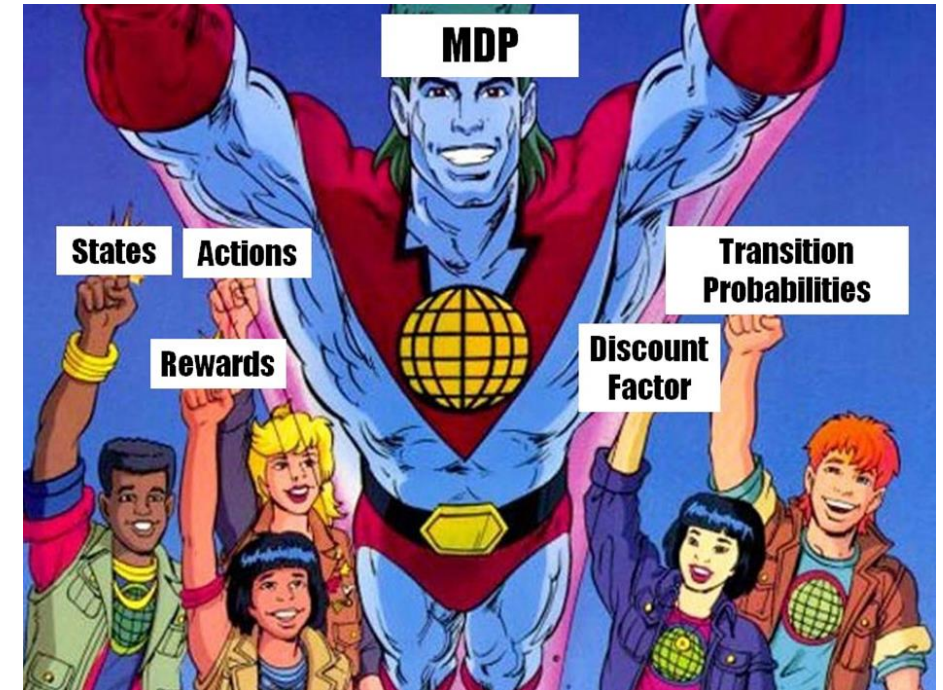


Assessment

Measure knowledge,
each student on their own,
stopped before success

Recap: Markov Decision Process (MDP)

- Definition: (S, A, T, R, γ)
 - A set of states $s \in S$
 - A set of actions $a \in A$
 - A transition function $T(s, a, s')$
 - Probability that a from s leads to s' , i.e., $P(s'|s, a)$
 - Also called the model or the dynamics
 - A reward function $R(s, a, s')$
 - A discount factor γ
- Quantities:
 - Policy: map of states to actions
 - Utility: sum of discounted rewards
 - State values: expected future utility from a state
 - State-action (Q-)values: expected future utility from a state-action pair



Recap: Value Functions

- Motivation: we don't know P or R



Which states/actions are good? $\rightarrow$ value function: $V^\pi(s)$ / $Q^\pi(s, a)$ [expected returns]

- State value function for policy π : the expected return starting from state s , and then following policy π

$$\underbrace{V^\pi(s)}_{\text{value of a state}} = \underbrace{\mathbb{E}_\pi[G_t | S_t = s]}_{\text{expected discounted return}} = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

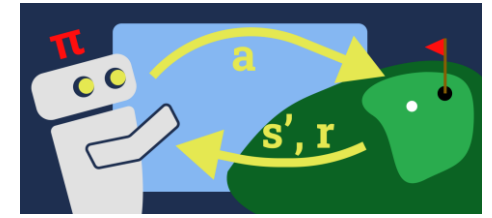
starting at state s

Note: the value of the terminal state (if any) is always zero

- State-action value function for policy π : the expected return starting from state s , taking action a , and then following policy π

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]$$

Recap: Value Functions



- Recursive relationships for returns

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots = R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \dots) = R_{t+1} + \gamma G_{t+1}$$



Condition on a state and take expectations

$$\mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s]$$

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1}) | S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma V^\pi(s')] \end{aligned}$$



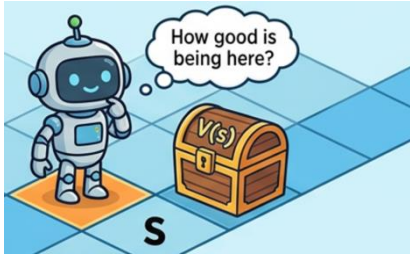
Condition on a state and action and take expectations

$$\mathbb{E}[G_t | S_t = s, A_t = a] = \mathbb{E}[R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a]$$

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}[R_{t+1} + \gamma Q^\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = a] \\ &= \sum_{s', r} p(s', r | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') Q^\pi(s', a') \right] \end{aligned}$$

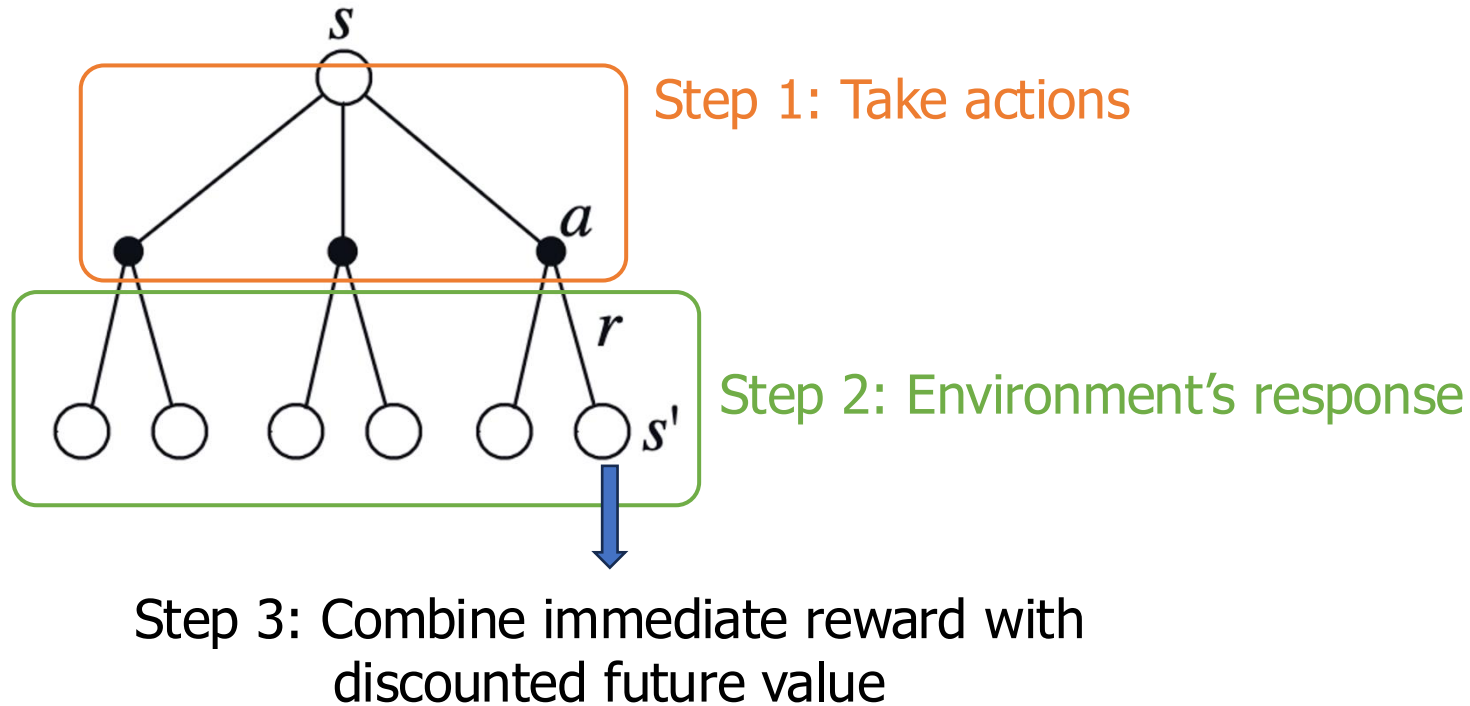
Bellman Expectation Equations

- Bellman equation for V^π :



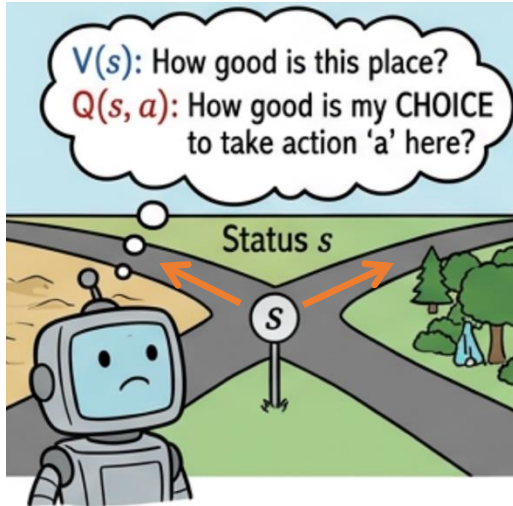
$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma V^\pi(s')]$$

- The value of state s is the expected discounted total reward from now on



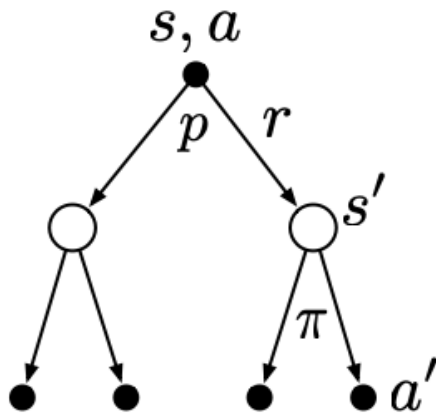
Bellman Expectation Equations

- Bellman equation for Q^π :



$$Q^\pi(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') Q^\pi(s', a') \right]$$

➤ $Q^\pi(s, a)$: the expected long-term value of taking a specific action a in current state s

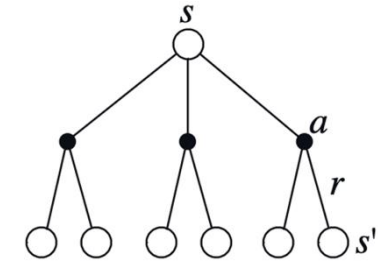


Bellman Expectation Equations

- Bellman equation for V^π :

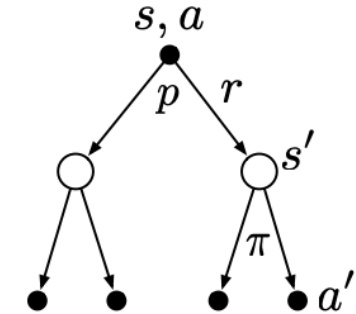
Relationship 

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V^\pi(s')]$$



- Bellman equation for Q^π :

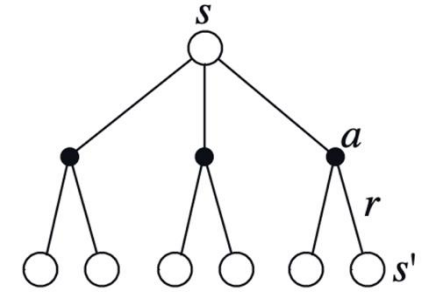
$$Q^\pi(s,a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \sum_{a'} \pi(a'|s') Q^\pi(s',a') \right]$$



Bellman Optimality Equations

- Bellman expectation equation for V^π :

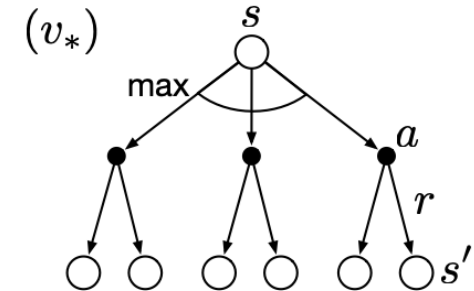
$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma V^\pi(s')]$$



- Bellman optimality equation for V^* :

- The value of a state under an optimal policy must equal the expected return for the **best** action from that state

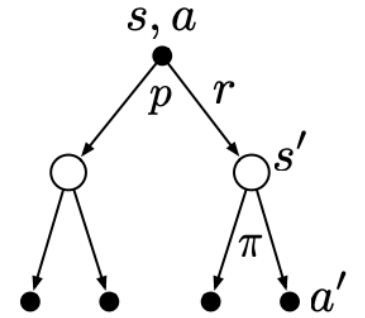
$$V^*(s) = \max_a \left(\sum_{s',r} p(s',r|s,a) [r + \gamma V^*(s')] \right)$$



Bellman Optimality Equations

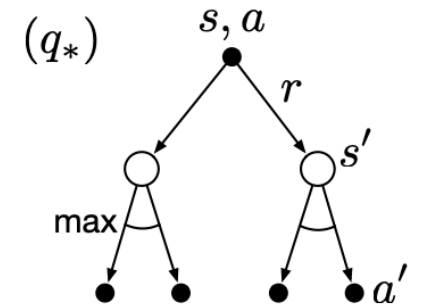
- Bellman expectation equation for Q^π :

$$Q^\pi(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \sum_{a'} \pi(a' | s') Q^\pi(s', a') \right]$$



- Bellman optimality equation for Q^* :

$$Q^*(s, a) = \sum_{s', r} p(s', r | s, a) \left[r + \gamma \max_{a'} Q^*(s', a') \right]$$



Bellman Optimality Equations

- Bellman optimality equation for V^* :

Relationship 

$$V^*(s) = \max_a \left(\sum_{s',r} p(s',r|s,a) [r + \gamma V^*(s')] \right)$$

- Bellman optimality equation for Q^* :

$$Q^*(s,a) = \sum_{s',r} p(s',r|s,a) \left[r + \gamma \max_{a'} Q^*(s',a') \right]$$

- Relationship: $V^*(s) = \max_a Q^*(s,a)$ $Q^*(s,a) = \sum_{s',r} p(s',r|s,a) [r + \gamma V^*(s')]$

- Link between value and policy: $\pi^*(s) = \operatorname{argmax}_a Q^*(s,a)$
 - Finding an optimal value function leads to having an optimal policy

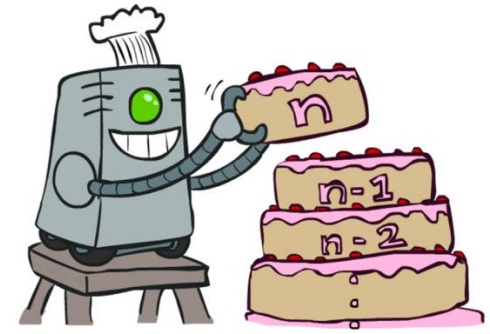
Value Iteration

- Bellman equation **characterizes** the optimal values

$$V^*(s) = \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V^*(s')]$$

- Value iteration **computes** the optimal values

$$V_{k+1}(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V_k(s')]$$



Recursion → Iteration

Value Iteration

- Value iteration **computes** the optimal values

$$V_{k+1}(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V_k(s')]$$

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

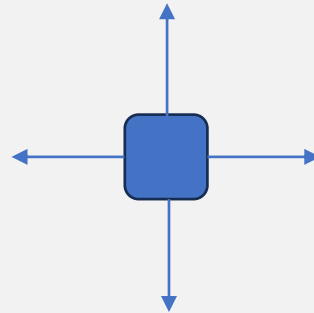
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$ 
| Loop for each  $s \in \mathcal{S}$ :
|    $v \leftarrow V(s)$ 
|    $V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$ 
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that

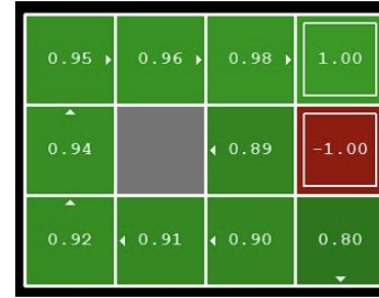
$$\pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$$



- The policy often converges long before the values
→ What about directly learning the policy?

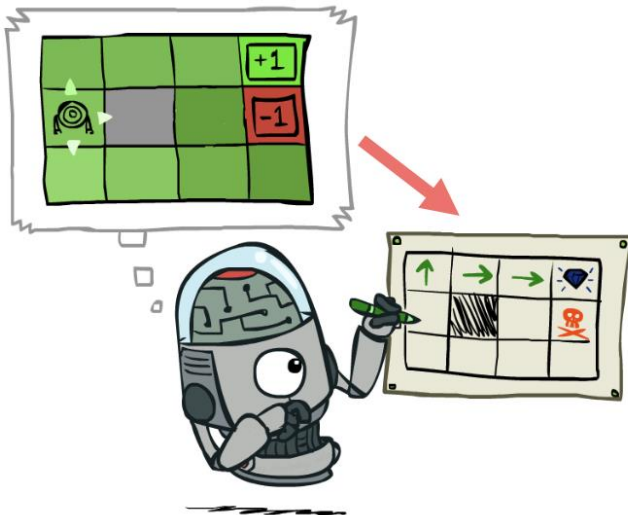
Computing Actions from Values

- Problem: how do we get actions?
 - Imagine we have the optimal values $V^*(s)$
 - How should we act?



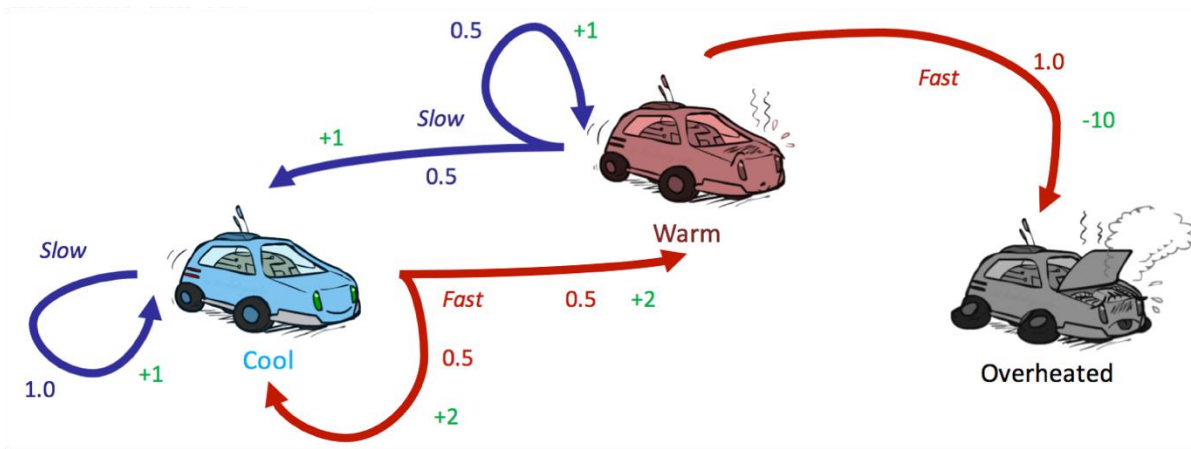
- Policy extraction: gets the policy implied by the values
 - We need to do a one-step lookup

$$\pi^*(s) = \operatorname{argmax}_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$



Value Iteration

- Example:
(assume discount=0.5)

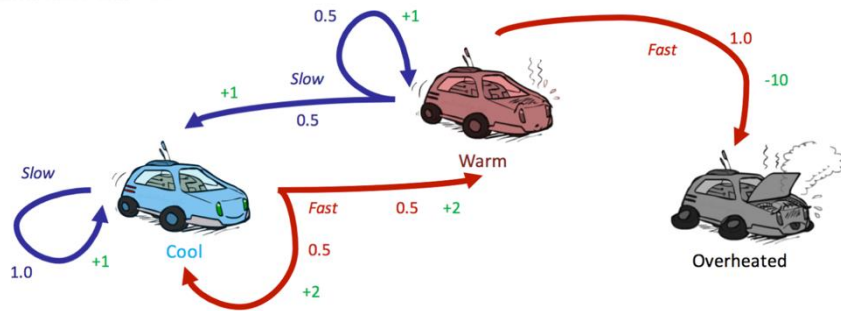


$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

s	a	s'	$P(s' s, a)$	$R(s, a, s')$
Cool	Slow	Cool	1.0	+1
Cool	Fast	Cool	0.5	+2
Cool	Fast	Warm	0.5	+2
Warm	Slow	Cool	0.5	+1
Warm	Slow	Warm	0.5	+1
Warm	Fast	Overheated	1.0	-10
Overheated	(end)	Overheated	1.0	0

Value Iteration

- Example: Car racing



s	a	s'	$P(s' s, a)$	$R(s, a, s')$
	Slow		1.0	+1
	Fast		0.5	+2
	Fast		0.5	+2
	Slow		0.5	+1
	Slow		0.5	+1
	Fast		1.0	-10
	(end)		1.0	0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')] \quad (\text{assume discount}=0.5)$$

We begin value iteration by initializing all $V_0(s) = 0$:

	cool	warm	overheated
V_0	0	0	0

In our first round of updates, we can compute $\forall s \in S, V_1(s)$ as follows:

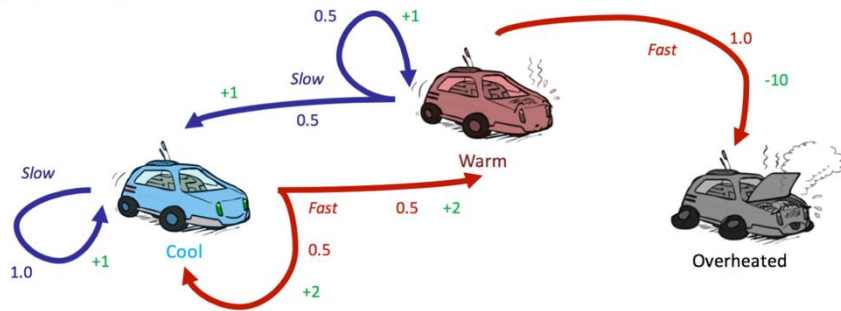
$$\begin{aligned} V_1(\text{cool}) &= \max\{1 \cdot [1 + 0.5 \cdot 0], 0.5 \cdot [2 + 0.5 \cdot 0] + 0.5 \cdot [2 + 0.5 \cdot 0]\} \\ &= \max\{1, 2\} \\ &= \boxed{2} \end{aligned}$$

$$\begin{aligned} V_1(\text{warm}) &= \max\{0.5 \cdot [1 + 0.5 \cdot 0] + 0.5 \cdot [1 + 0.5 \cdot 0], 1 \cdot [-10 + 0.5 \cdot 0]\} \\ &= \max\{1, -10\} \\ &= \boxed{1} \end{aligned}$$

$$\begin{aligned} V_1(\text{overheated}) &= \max\{\} \\ &= \boxed{0} \end{aligned}$$

Value Iteration

- Example: Car racing



s	a	s'	$P(s' s, a)$	$R(s, a, s')$
 Cool	 Slow	 Cool	1.0	+1
 Cool	 Fast	 Cool	0.5	+2
 Cool	 Fast	 Warm	0.5	+2
 Warm	 Slow	 Cool	0.5	+1
 Warm	 Slow	 Warm	0.5	+1
 Warm	 Fast	 Overheated	1.0	-10
 Overheated	(end)	 Overheated	1.0	0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')] \quad (\text{assume discount}=0.5)$$

	cool	warm	overheated
V_0	0	0	0
V_1	2	1	0

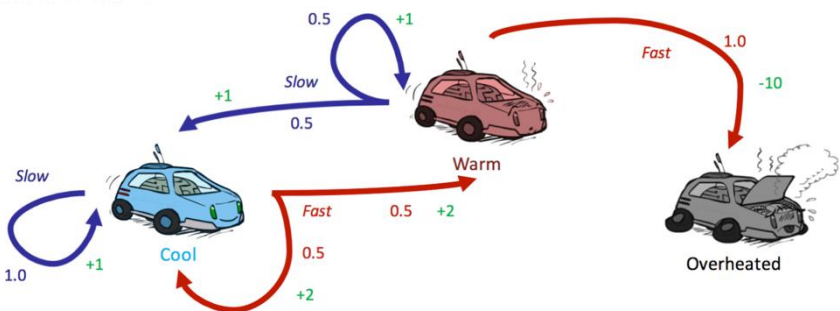
$$\begin{aligned} V_2(\text{cool}) &= \max\{1 \cdot [1 + 0.5 \cdot 2], 0.5 \cdot [2 + 0.5 \cdot 2] + 0.5 \cdot [2 + 0.5 \cdot 1]\} \\ &= \max\{2, 2.75\} \\ &= \boxed{2.75} \end{aligned}$$

$$\begin{aligned} V_2(\text{warm}) &= \max\{0.5 \cdot [1 + 0.5 \cdot 2] + 0.5 \cdot [1 + 0.5 \cdot 1], 1 \cdot [-10 + 0.5 \cdot 0]\} \\ &= \max\{1.75, -10\} \\ &= \boxed{1.75} \end{aligned}$$

$$\begin{aligned} V_2(\text{overheated}) &= \max\{\} \\ &= \boxed{0} \end{aligned}$$

Value Iteration

- Example: Car racing



<i>s</i>	<i>a</i>	<i>s'</i>	<i>P(s' s, a)</i>	<i>R(s, a, s')</i>
 Cool	 Slow	 Cool	1.0	+1
 Cool	 Fast	 Cool	0.5	+2
 Cool	 Fast	 Warm	0.5	+2
 Warm	 Slow	 Cool	0.5	+1
 Warm	 Slow	 Warm	0.5	+1
 Warm	 Fast	 Overheated	1.0	-10
 Overheated	(end)	 Overheated	1.0	0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')] \quad (\text{assume discount}=0.5)$$

	cool	warm	overheated
V_0	0	0	0
V_1	2	1	0
V_2	2.75	1.75	0

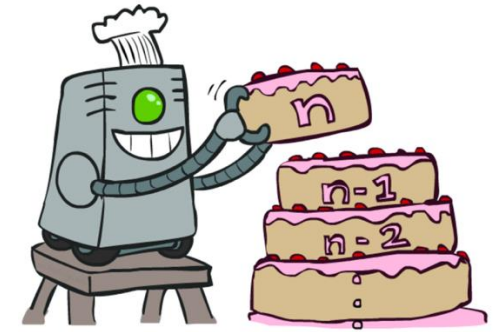
Value Iteration

- Bellman equation **characterizes** the optimal values

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

- Value iteration **computes** the optimal values

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$



Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

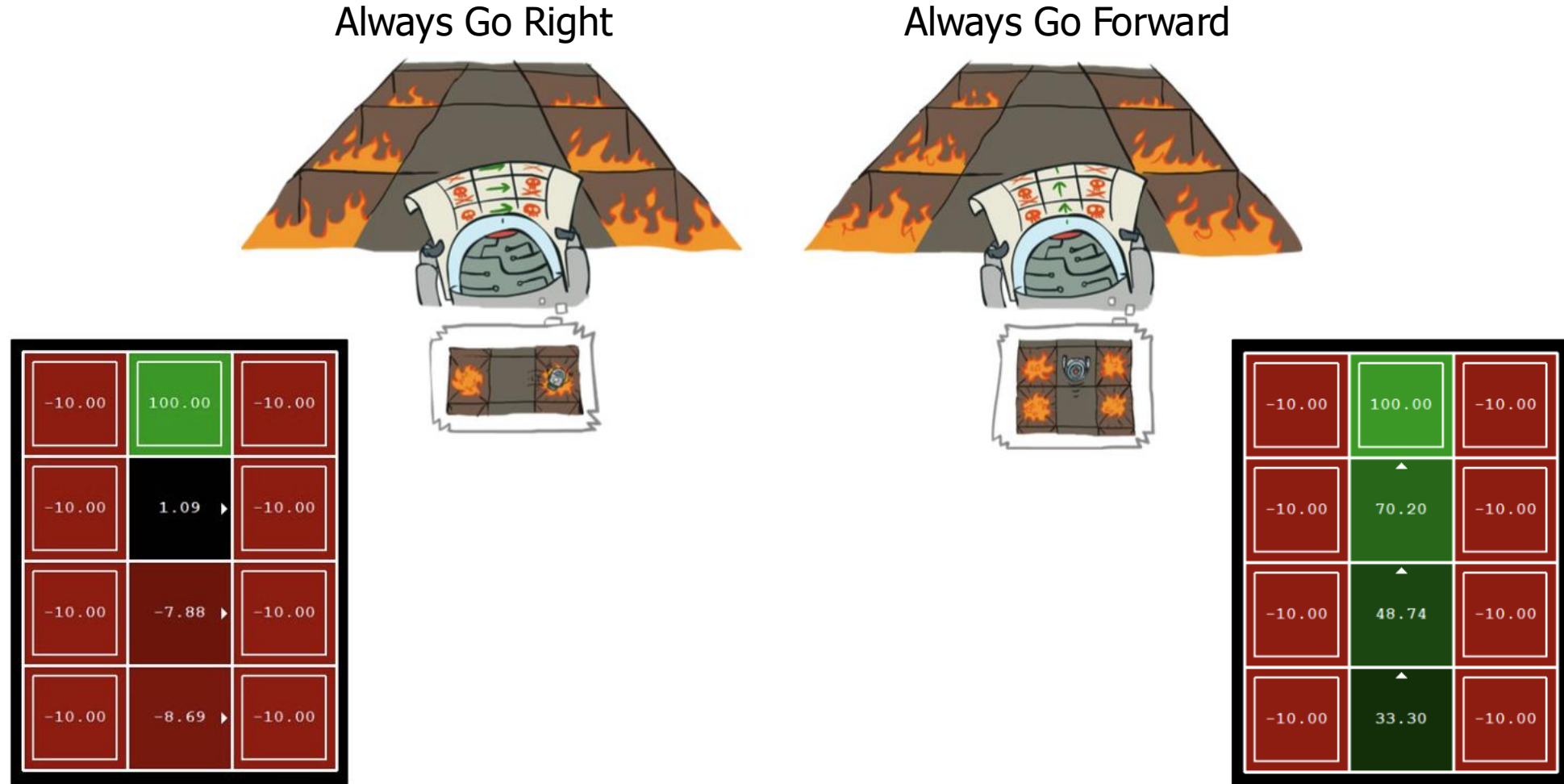
Loop:

```
|  $\Delta \leftarrow 0$   
| Loop for each  $s \in \mathcal{S}$ :  
|    $v \leftarrow V(s)$   
|    $V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$   
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$   
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that
 $\pi(s) = \arg\max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$

- Problem 1: slow – $O(|\mathcal{S}|^2 |A|)$ per iteration
- Problem 2: The policy often converges long before the values

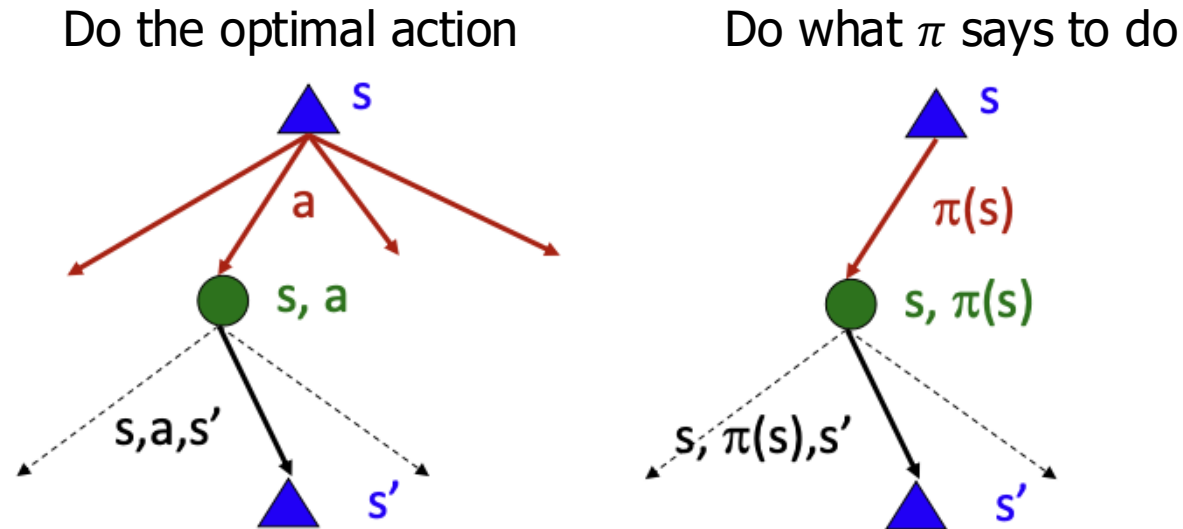
Policy Iteration-Part 1: Policy Evaluation



Usage: computing the value function for a fixed policy is to help find better policies.

Fixed Policies

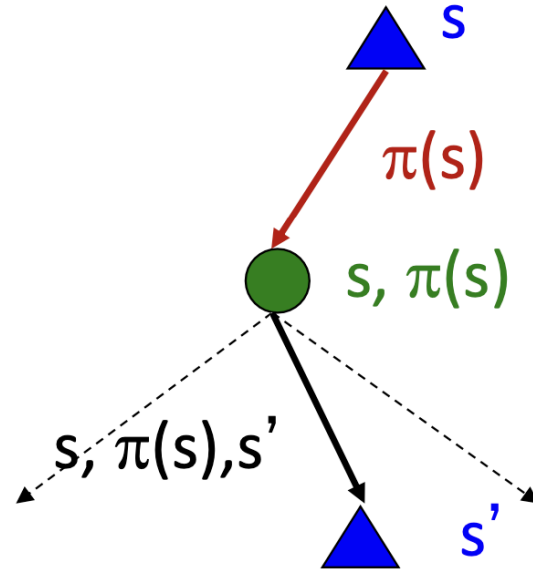
- Value iteration max over all actions to compute the optimal values
- If we fixed some policy $\pi(s)$, then the tree would be simpler – only one action per state
 - though the tree's value would depend on which policy we fixed



Utilities for a Fixed Policy

- Define the utility of a state s , under a fixed policy π :

$V^\pi(s)$ = expected total discounted rewards starting in s and following π



Utilities for a Fixed Policy

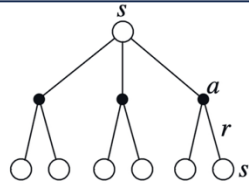
- Define the utility of a state s , under a fixed policy π :

$V^\pi(s)$ = expected total discounted rewards starting in s and following π

- How do we calculate the value function for a fixed policy π ?
 - Bellman expectation equation (recursive relation)

- Bellman expectation equation for V^π :

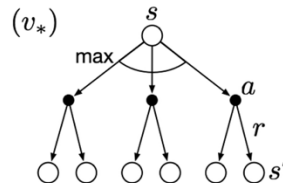
$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma V^\pi(s')]$$



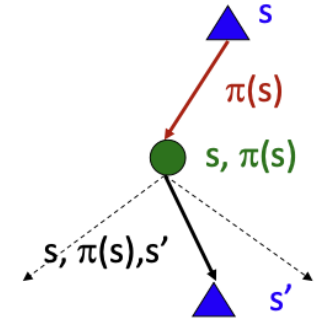
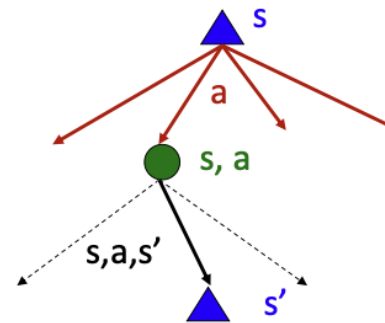
- Bellman optimality equation for V^* :

- The value of a state under an optimal policy must equal the expected return for the **best** action from that state

$$V^*(s) = \max_a \left(\sum_{s',r} p(s',r|s,a)[r + \gamma V^*(s')] \right)$$



Do the optimal action Do what π says to do



Policy Evaluation

- How do we calculate the value function for a fixed policy π ?
 - Idea 1: Turn recursive Bellman equations into updates (like value iteration)

$$V_0^\pi(s) = 0$$

$$V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V_k^\pi(s')]$$

- Efficiency: $O(|S|^2)$ per iteration
- Idea 2: Without the maxes, the Bellman equations are just a linear system
 - Solve with your favorite linear system solver (e.g., Matlab)

$$\begin{bmatrix} & & \\ & & \\ & & \end{bmatrix} \mathbf{X} \begin{bmatrix} V^\pi(s_1) \\ V^\pi(s_2) \\ \dots \end{bmatrix} = \begin{bmatrix} \\ \\ \end{bmatrix}$$

Policy Evaluation

- How do we calculate the value function for a fixed policy π ?
 - Bellman expectation equation

• Bellman expectation equation for V^π :

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma V^\pi(s')]$$


• Bellman optimality equation for V^* :

- The value of a state under an optimal policy must equal the expected return for the **best** action from that state

$$V^*(s) = \max_a \left(\sum_{s',r} p(s',r|s,a)[r + \gamma V^*(s')] \right)$$

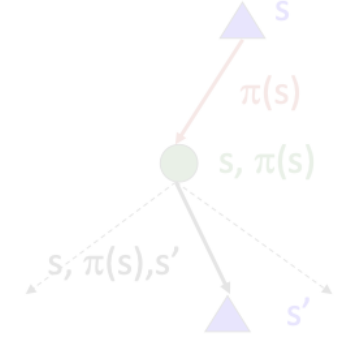
(v_*)



Do the optimal action



Do what π says to do



- Turn recursive Bellman expectation equations into updates (like value iteration)

$$V_0^\pi(s) = 0$$

$$V_{k+1}^\pi(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma V_k^\pi(s')]$$

Policy Evaluation

- How do we calculate the value function for a fixed policy π ?
 - Turn recursive Bellman expectation equations into updates (like value iteration)
→ Iterative Policy Evaluation

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π , the policy to be evaluated

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma v_\pi(s')]$$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$ arbitrarily, for $s \in \mathcal{S}$, and $V(\text{terminal})$ to 0

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

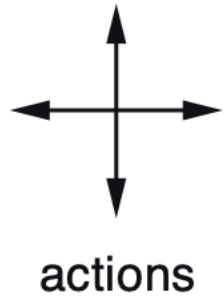
$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

Policy Evaluation

- Example: Consider the 4x4 GridWorld shown below.
 - An undiscounted episodic task
 - Non-terminal states: $\{1, \dots, 14\}$; Terminal state: lower right corner
 - Actions: $\{\uparrow, \downarrow, \leftarrow, \rightarrow\}$
 - Transition dynamics: actions that would take the agent off the grid leave the state unchanged
 - Rewards: -1 until the terminal state is reached



	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$R_t = -1$
on all transitions

Policy Evaluation

- Example: Consider the 4x4 GridWorld shown below.

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

$k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0

ELEC6910J

Deep Reinforcement Learning

Lectures 7. Dynamic Programming (II) Policy Improvement, Policy Iteration

Course Instructor: Ling PAN

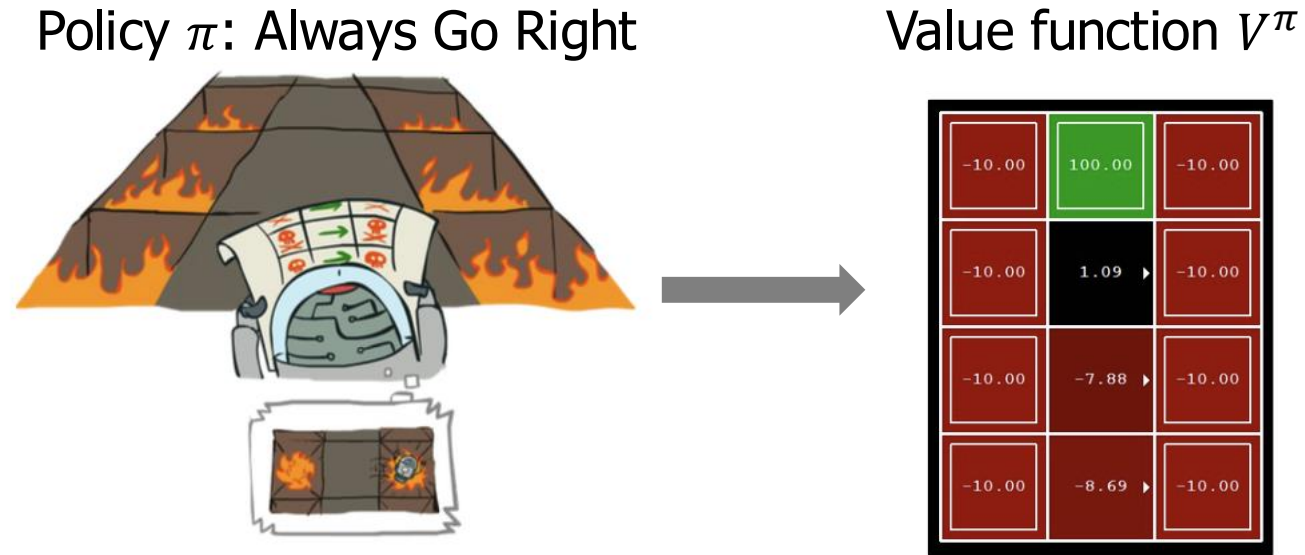
Department of Electronic and Computer Engineering

Hong Kong University of Science and Technology

Most slides are adapted from UC Berkeley CS188/CS285, CMU 10-403

Policy Iteration-Part 2: Policy Improvement

- Suppose we have computed V^π for a deterministic policy π



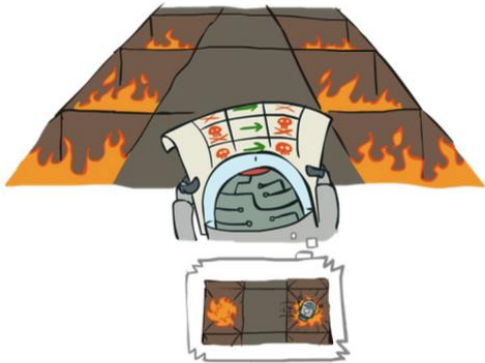
- Question: Can we do better? / How can we do better?

Policy Improvement

- For a given state s ,
 - would it be better to do an action $a \neq \pi(s)$ and thereafter follow π ?
- We know how good it is to follow the current policy π from $s \rightarrow V^\pi(s)$
- Would it be better or worse to change to a new policy?

Policy π : Always Go Right

Value function V^π



-10.00	100.00	-10.00
-10.00	1.09	-10.00
-10.00	-7.88	-10.00
-10.00	-8.69	-10.00

New policy:

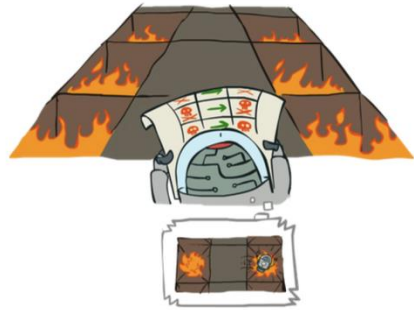
- Go forward
- Follow π (always go right) thereafter



Policy Improvement

- Given a policy and its value function, we can easily evaluate a change in the policy at a single state.

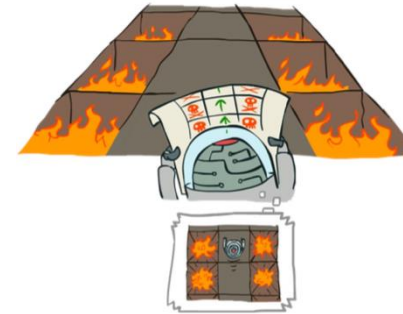
Policy π : Always Go Right



Value function V^π

-10.00	100.00	-10.00
-10.00	1.09	-10.00
-10.00	-7.88	-10.00
-10.00	-8.69	-10.00

The core idea: testing a new action



New policy:

- Go forward
- Follow π (always go right) thereafter

Policy Improvement

- A potential new policy: switch to action a for state s if and only if $Q^\pi(s, a) > V^\pi(s)$
 - Meaning: it is better to select a once in s and thereafter follow π than it would be to follow π all the time
 - The value of this way of behaving:

$$\begin{aligned} q_\pi(s, a) &\doteq \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')]. \end{aligned}$$

- Key criterion:

$$Q^\pi(s, a) \begin{matrix} > \\ < \\ = \end{matrix} V^\pi(s)$$

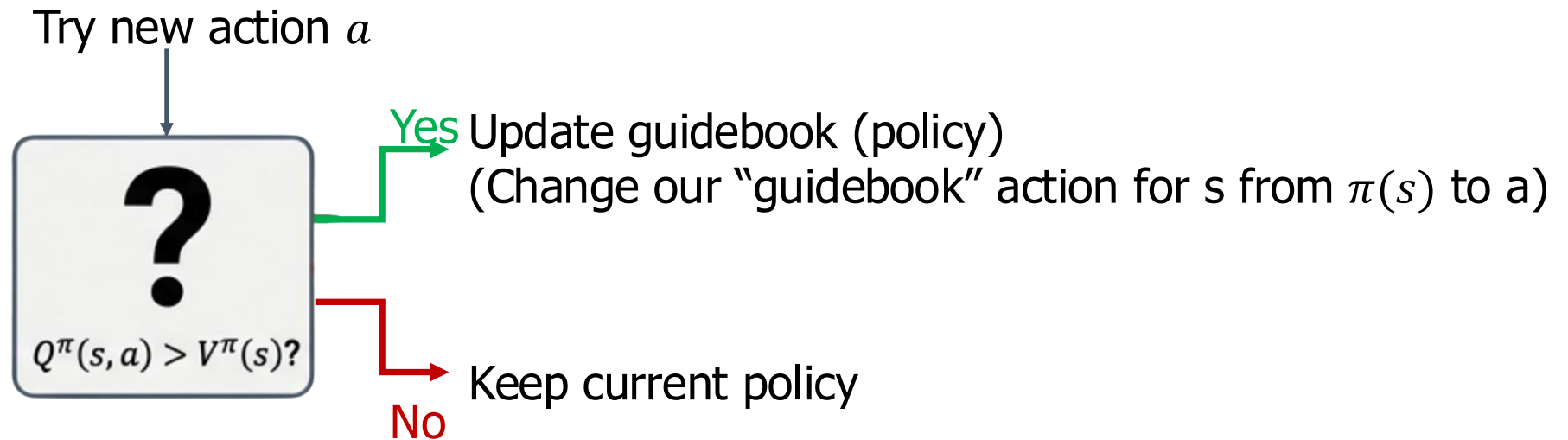

- If $Q^\pi(s, a) > V^\pi(s)$:
 - Better to select a every time s is encountered
 - The new policy would in fact be a better one overall

Policy Improvement

- Our new policy:

switch to action a for state s if and only if $Q^\pi(s, a) > V^\pi(s)$

- Meaning: it is better to select a once in s and thereafter follow π than it would be to follow π all the time
- ✓ Theoretical guarantee: policy improvement theorem



Policy Improvement

- Policy improvement theorem:

- Let π and π' be any pair of deterministic policies such that, for all $s \in S$

$$Q^\pi(s, \pi'(s)) \geq V^\pi(s), \quad (1)$$

then the policy π' must be as good as, or better than π .

That is, it must obtain greater or equal expected return from all states $s \in S$:

$$V^{\pi'}(s) \geq V^\pi(s). \quad (2)$$

If there is strict inequality of Eq. (1) at any state, then there must be strict inequality of Eq. (2) at that state.

Policy Improvement

- Given a policy and its value function, we can easily evaluate a change in the policy at a single state.
- Extension: consider changes at all states, selecting at each state the action that appears best according to $q_\pi(s, a)$

- New greedy policy:

$$\begin{aligned}\pi'(s) &\doteq \arg\max_a q_\pi(s, a) \quad (\text{ties broken arbitrarily}) \\ &= \arg\max_a \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \arg\max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')],\end{aligned}$$

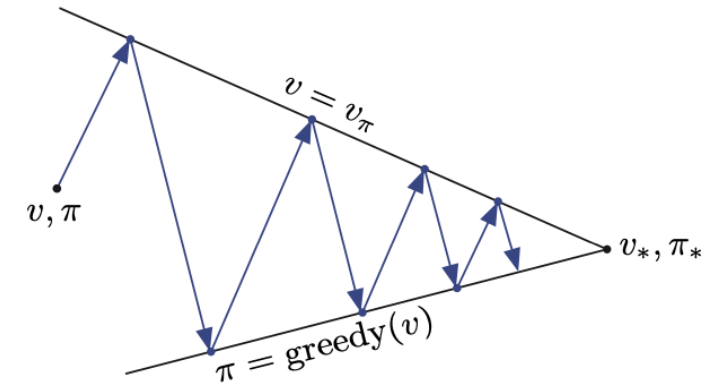
- Meaning: takes the action that looks best in the short term — after one step of lookahead according to v_π
- Effect: the greedy policy meets the conditions of the policy improvement theorem by construction, so we know that it is as good as, or better than, the original policy

Policy Improvement

- Summary: The process of making a new policy that improves on an original policy, by making it greedy with respect to the value function of the original policy, is called policy improvement.
- Policy improvement thus must give us a strictly better policy except when the original policy is already optimal.
- Note: all the ideas of this section extend easily to stochastic policies.

Policy Iteration

- Alternative approach for optimal values:
 - **Step 1 - Policy evaluation:** calculate utilities for some fixed policy (not optimal utilities!) until convergence
 - **Step 2 - Policy improvement:** update policy using one-step look-ahead with resulting converged (but not optimal!) utilities as future values
 - Repeat steps until policy converges
- This is **policy iteration**
 - ✓ Convergence guarantee
 - It's still optimal!
 - Can converge (much) faster under some conditions



$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \dots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*$$

Policy Iteration

- Evaluation: For fixed current policy π , find values with policy evaluation:
 - Iterate until values converge:

$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_a \pi_i(a|s) \sum_{s',r} p(s',r|s,a)[r + \gamma V_k^{\pi_i}(s')]$$

- End up with value function
- Improvement: For fixed values, get a better policy using policy extraction
 - One-step look-ahead:

$$\pi_{i+1}(s) = \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a)[r + \gamma V^{\pi_i}(s')]$$

- Repeat steps until policy converges

Policy Iteration

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$; $V(\text{terminal}) \doteq 0$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

$\text{policy-stable} \leftarrow \text{true}$

For each $s \in \mathcal{S}$:

$\text{old-action} \leftarrow \pi(s)$

$\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

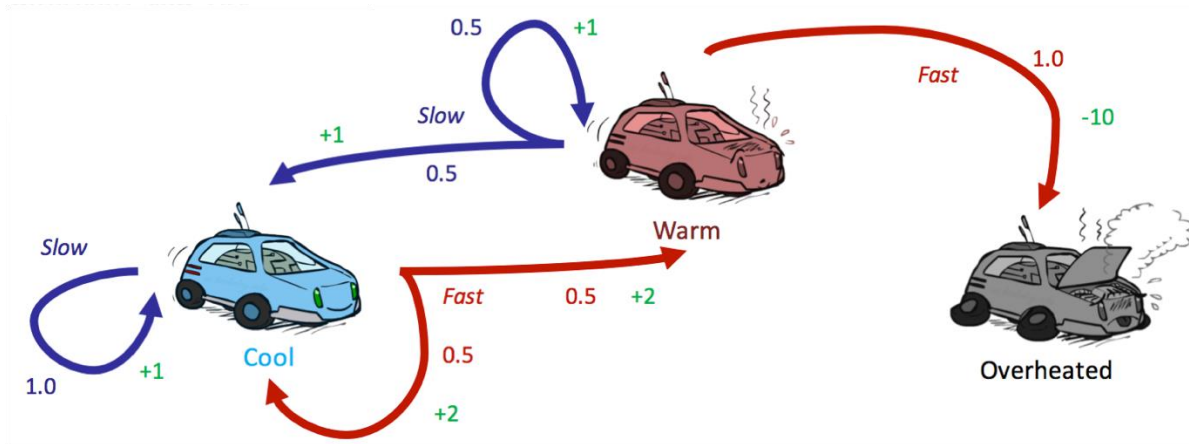
If $\text{old-action} \neq \pi(s)$, then $\text{policy-stable} \leftarrow \text{false}$

If policy-stable , then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

[Demo](#)

Policy Iteration

- Example:
(assume discount=0.5)



s	a	s'	$P(s' s, a)$	$R(s, a, s')$
	Slow		1.0	+1
	Fast		0.5	+2
	Fast		0.5	+2
	Slow		0.5	+1
	Slow		0.5	+1
	Fast		1.0	-10
	(end)		1.0	0

We start with an initial policy of *Always go slow*:

	cool	warm	overheated
π_0	slow	slow	—

$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T(s, \pi_i(s), s') [R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s')]$$

$$\pi_{i+1}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_i}(s')]$$

As terminal states have no outgoing actions, no policy can assign a value to one.
Hence, we just disregard the state *overheated* from consideration (simply assign $V^{\pi_i}(s) = 0, \forall i$ for any terminal states s).

Policy Iteration

- Example: (assume discount=0.5)

s	a	s'	$P(s' s, a)$	$R(s, a, s')$
 Cool	 Slow	 Cool	1.0	+1
 Cool	 Fast	 Cool	0.5	+2
 Cool	 Fast	 Warm	0.5	+2
 Warm	 Slow	 Cool	0.5	+1
 Warm	 Slow	 Warm	0.5	+1
 Warm	 Fast	 Overheated	1.0	-10
 Overheated	(end)	 Overheated	1.0	0

$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T(s, \pi_i(s), s') [R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s')] \quad \pi_{i+1}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^{\pi_i}(s')]$$

$$\begin{aligned} V^{\pi_0}(\text{cool}) &= 1 \cdot [1 + 0.5 \cdot V^{\pi_0}(\text{cool})] \\ V^{\pi_0}(\text{warm}) &= 0.5 \cdot [1 + 0.5 \cdot V^{\pi_0}(\text{cool})] \\ &\quad + 0.5 \cdot [1 + 0.5 \cdot V^{\pi_0}(\text{warm})] \end{aligned}$$

	cool	warm	overheated
π_0	slow	slow	—

Solving this system of equations for $V^{\pi_0}(\text{cool})$ and $V^{\pi_0}(\text{warm})$ yields:

	cool	warm	overheated
V^{π_0}	2	2	0

We can now run policy extraction with these values:

$$\begin{aligned} \pi_1(\text{cool}) &= \operatorname{argmax}\{\text{slow} : 1 \cdot [1 + 0.5 \cdot 2], \text{fast} : 0.5 \cdot [2 + 0.5 \cdot 2] + 0.5 \cdot [2 + 0.5 \cdot 2]\} \\ &= \operatorname{argmax}\{\text{slow} : 2, \text{fast} : 3\} \\ &= \boxed{\text{fast}} \end{aligned}$$

$$\begin{aligned} \pi_1(\text{warm}) &= \operatorname{argmax}\{\text{slow} : 0.5 \cdot [1 + 0.5 \cdot 2] + 0.5 \cdot [1 + 0.5 \cdot 2], \text{fast} : 1 \cdot [-10 + 0.5 \cdot 0]\} \\ &= \operatorname{argmax}\{\text{slow} : 2, \text{fast} : -10\} \\ &= \boxed{\text{slow}} \end{aligned}$$

	cool	warm
π_0	slow	slow
π_1	fast	slow

Policy Iteration

- Example: (assume discount=0.5) $V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T(s, \pi_i(s), s') [R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s')]$ $\pi_{i+1}(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k^{\pi_i}(s')]$

s	a	s'	$P(s' s, a)$	$R(s, a, s')$
 Cool	 Slow	 Cool	1.0	+1
 Cool	 Fast	 Cool	0.5	+2
 Cool	 Fast	 Warm	0.5	+2
 Warm	 Slow	 Cool	0.5	+1
 Warm	 Slow	 Warm	0.5	+1
 Warm	 Fast	 Overheated	1.0	-10
 Overheated	(end)	 Overheated	1.0	0

Running policy iteration for a second round yields $\pi_2(\text{cool}) = \text{fast}$ and $\pi_2(\text{warm}) = \text{slow}$. Since this is the same policy as π_1 , we can conclude that $\pi_1 = \pi_2 = \pi^*$.

	cool	warm
π_0	slow	slow
π_1	fast	slow
π_2	fast	slow

This example shows the true power of policy iteration:
with only two iterations, we have already arrived at the optimal policy for our racecar MDP!

Convergence

- What we just learned is actually trying to find $f(x) = x$ (fixed point problem)
 - Claim: we will ultimately find a value x^* as the solution
- Question:
 - How do we know that value iteration converges to v_* ?
 - Or that iterative policy evaluation converges to v_π ? And therefore that policy iteration converges to v_* ?
 - Is the solution unique?
 - How fast do these algorithms converge?
- Answer: Contraction Mapping Theorem

Convergence

- Value Function Space
 - Consider the vector space V over value functions
 - There are $|S|$ dimensions
 - Each point in this space fully specifies a value function $v(s)$
 - What does a Bellman backup do to points in this space?
 - We will show that it brings value functions **closer**
 - And therefore the backups must converge on a unique solution

Value Function ∞ -Norm

- We will measure distance between state-value functions u and v by the ∞ -norm

$$\|u - v\|_{\infty} = \max_s |u(s) - v(s)|$$

↑
the largest difference between state values
(element-wise max)

Convergence: Bellman Expectation Backup is a Contraction

- Define the Bellman expectation backup operator T^π

$$T^\pi(v) = \mathcal{R}^\pi + \gamma \mathcal{P}^\pi v$$

- This operator is a γ -contraction, which makes value functions closer by at least γ

$$\begin{aligned} \|T^\pi(u) - T^\pi(v)\|_\infty &= \|(\mathcal{R}^\pi + \gamma \mathcal{P}^\pi u) - (\mathcal{R}^\pi + \gamma \mathcal{P}^\pi v)\|_\infty \\ &= \|\gamma \mathcal{P}^\pi(u - v)\|_\infty \\ &\leq \|\gamma \mathcal{P}^\pi\| \|u - v\|_\infty \\ &\leq \gamma \|u - v\|_\infty \end{aligned}$$

Convergence: Iterative Policy Evaluation and Policy Iteration

Contraction Mapping Theorem

For any metric space V that is complete (i.e., closed) under an operator $T(v)$, where T is a γ -contraction, then

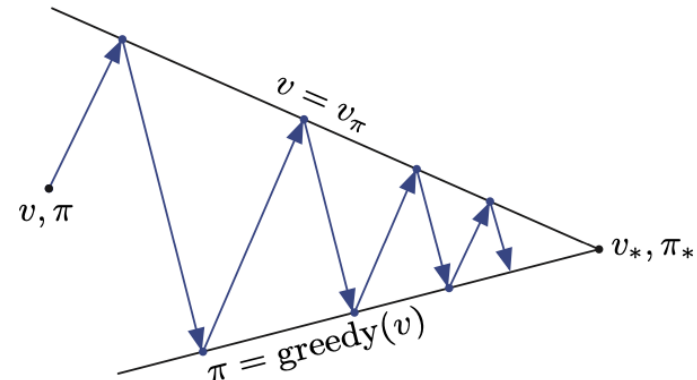
- T converges to a unique fixed point
- At a linear convergence rate of γ



- The Bellman expectation operator T^π has a unique fixed point
- v_π is a fixed point of T^π (by Bellman expectation equation $v_\pi = T^\pi(v_\pi)$)

- Therefore,
 - Iterative policy evaluation converges on v_π
 - Policy iteration converges on v_*

Inevitably, the joint process is brought closer to the overall goal of optimality.



Convergence: Bellman Optimality Backup is a Contraction

- Define the Bellman optimality backup operator T^*

$$T^*(v) = \max_{a \in \mathcal{A}} \mathcal{R}^a + \gamma \mathcal{P}^a v$$

- This operator is a γ -contraction, which makes value functions closer by at least γ

$$\|T^*(u) - T^*(v)\|_\infty \leq \gamma \|u - v\|_\infty$$

Convergence: Value Iteration

Contraction Mapping Theorem

For any metric space V that is complete (i.e., closed) under an operator $T(v)$, where T is a γ -contraction, then

- T converges to a unique fixed point
- At a linear convergence rate of γ



- The Bellman optimality operator T^* has a unique fixed point
- v_* is a fixed point of T^* (by Bellman optimality equation)
- Therefore,
 - Value iteration converges on v_*

Comparison

- Both value iteration and policy iteration compute the same thing (all optimal values)
- In value iteration:
 - Every iteration updates both the values and (implicitly) the policy
 - We don't track the policy, but taking the max over actions implicitly recomputes it
- In policy iteration:
 - We do several passes that update utilities with fixed policy (each pass is fast because we consider only one action, not all of them)
 - After the policy is evaluated, a new policy is chosen (slow like a value iteration pass)
 - The new policy will be better (or we're done)
- Both are dynamic programs for solving MDPs

Summary

- Bellman equation zoo

- Optimal V and Q value functions

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \max_{a'} Q^*(s', a')]$$



$$V^*(s) = \max_a Q^*(s, a)$$

- Value function for fixed policy π

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')]$$

- Policy π for V and Q value functions

$$\pi^*(s) = \operatorname{argmax}_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

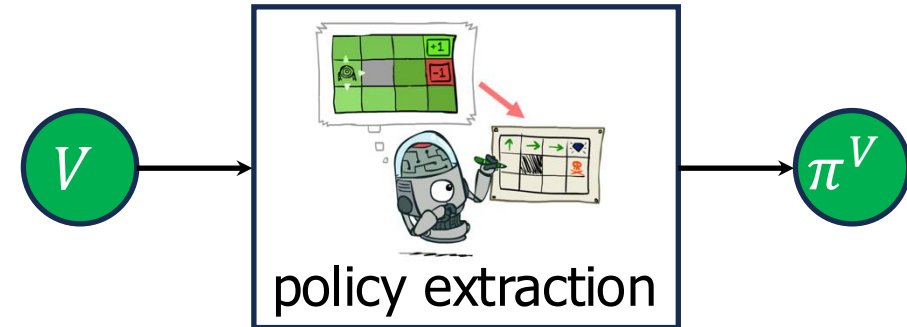
$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$$

Summary

- If you want to
 - Compute optimal values: use value iteration or policy iteration



- Compute values for a particular policy:
use policy evaluation
- Turn your values into a policy:
use policy extraction (one-step lookahead)



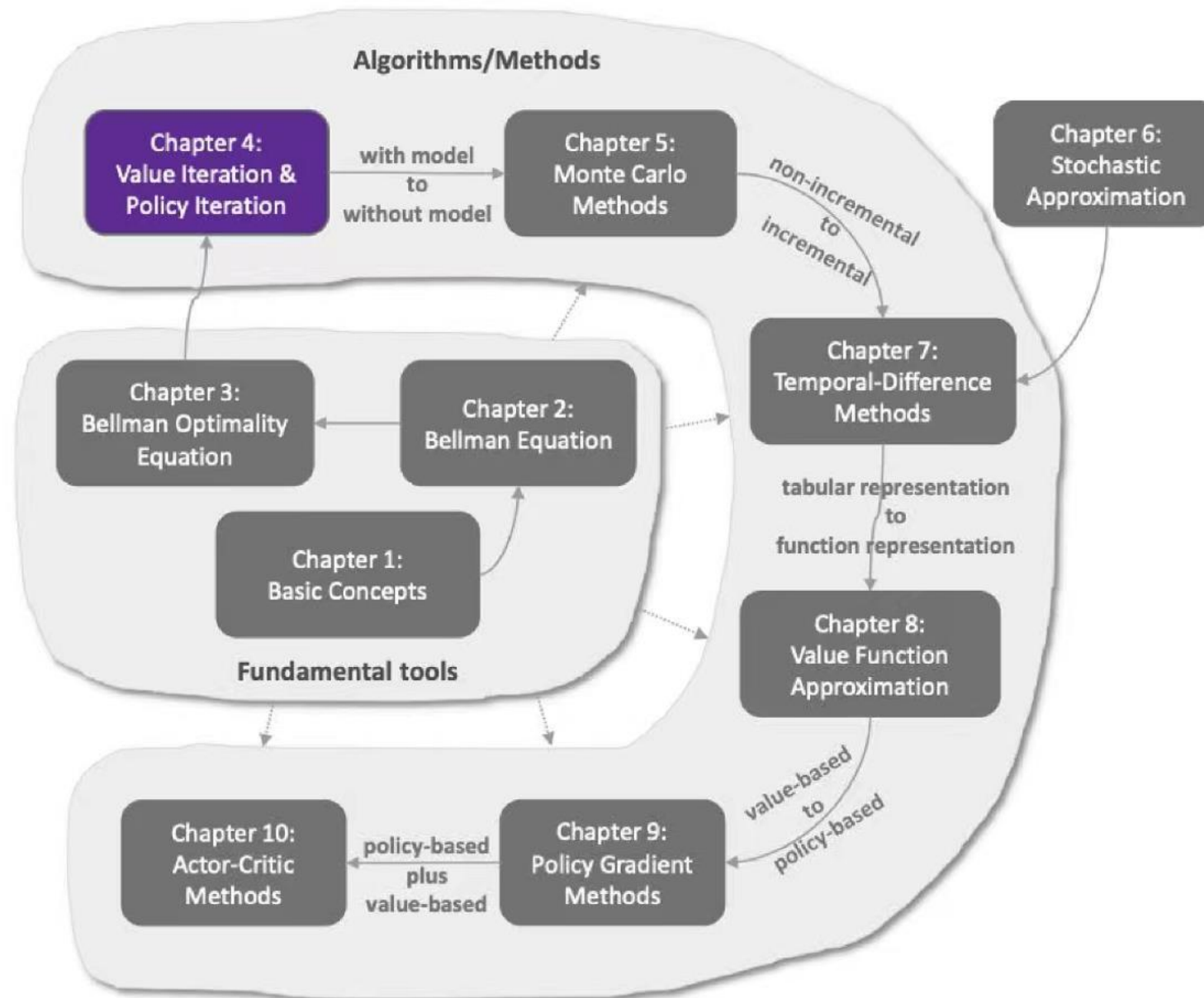
Problem

- So far, to estimate value functions we have been using dynamic programming with **known rewards and dynamics functions**:

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V^\pi(s')]$$

- Next: estimate value functions and policies from interaction experience, **without** known rewards or dynamics.
- How? By sampling all the way. Instead of probabilities distributions to compute expectations, we will use empirical expectations by averaging sampled returns.



Thanks!